SOFTWARE—PRACTICE AND EXPERIENCE
Softw. Pract. Exper. 2000; **30**:617–638

Introspective sorting and selection revisited

John D. Valois*,†

Department of Computer Science, Rensselaer Polytechnic Institute, Troy, NY 12180, U.S.A.

SUMMARY

We describe two improvements to introspective sorting and selection algorithms: a simple rule for *fine-grained introspection* that detects potential worst-case performance after only a small constant number of partitioning steps, and the use of *remedial randomization* as an intervention strategy in order to reduce the performance penalty for false positives. We present experimental results showing that these techniques provide significant improvements in the running time for worst-case and other troublesome inputs, without sacrificing performance on well-behaved inputs. Copyright © 2000 John Wiley & Sons, Ltd.

KEY WORDS: Quicksort; Find; sorting and selection algorithms; introspective algorithms; hybrid algorithms; generic algorithms

INTRODUCTION

Quicksort using the median-of-3 pivot selection strategy [1,2] is considered the preferred method to sort an array in place, being several times faster than Heapsort in practice [3]. The companion Find algorithm is similarly preferred for determining the mth ranked element in a sequence, and is likewise faster in practice than the linear time selection algorithm of Blum *et al.* [4]. Both have been widely implemented. However, it is well known that in the worst case both algorithms require quadratic time, compared with $\Theta(n \log n)$ and $\Theta(n)$, respectively, in the average case.

Musser [3] introduced the idea of *introspective* versions of these algorithms that monitor their own performance and take remedial action to avoid quadratic behavior when a potential worst-case input is detected. *Introsort*, Musser's introspective Quicksort variant, monitors the depth of its recursion and assumes a worst-case input if this exceeds its expected value by too much; in such cases Introsort switches to Heapsort as a 'stopper' algorithm. Musser suggests a similar approach for selection, using the algorithm of Blum *et al.* as the stopper. The resulting hybrid algorithms are guaranteed to run

*Correspondence to: John D. Valois, Department of Computer Science, Rensselaer Polytechnic Institute, Troy, NY 12180, U.S.A.
†E-mail: valoisj@cs.rpi.edu

Copyright © 2000 John Wiley & Sons, Ltd.

Received 30 September 1999
Revised 27 December 1999
Accepted 28 December 1999

in time $\Theta(n \log n)$ and $\Theta(n)$, respectively, while retaining the small constant factors and excellent performance of standard Quicksort and Find for most inputs.

Because a significant amount of processing (a logarithmic number of partitioning steps) occurs before Introsort switches to the stopper algorithm, a performance penalty is incurred on worst-case inputs, for which Introsort requires about twice as much time as Heapsort [3]. There is an even larger performance penalty for inputs that generate false positives; if an input is not a true worst case but causes the depth of recursion to exceed its expected value by too much (e.g., Musser's 'two-faced' sequence), Introsort is not only slower than Heapsort, but much slower than standard non-introspective Quicksort.

In this paper we describe two techniques that eliminate this performance penalty. The first is a simple heuristic for *fine-grained introspection* that detects potential worst-case inputs after a small constant number of partitioning steps. Increasing the granularity of the introspection permits *remedial randomization*; rather than switching to the stopper algorithm when a potential worst case is detected, a small amount of entropy is injected into the input. This will often perturb the true worst-case inputs sufficiently so that performance reverts to the average case, while resulting in practically no penalty for false positives. We present experimental results showing that our new algorithms handle many difficult inputs within a factor of two to three times the performance of the standard algorithm on random inputs, while introducing virtually no overhead on best-case and random inputs.

FINE-GRAINED INTROSPECTION

Introsort works by monitoring the depth of recursion, requiring $2\lfloor \log_2 n \rfloor$ partitioning steps to identify a potential worst-case input. It turns out that we can reduce this number to a small constant by basing the introspection, instead, on the number of steps required to reduce the size of the array under consideration by one half. This metric was originally suggested [3] for introspective selection algorithms.

We now show that a simple linear function of the form $f(x) = \lfloor mx \rfloor + b$ gives, within a reasonable range for x, the expected number of partitioning steps required to reduce the size of the array to less than 2^{-x} times its original size. The choice of the constant b determines the *sensitivity* of the algorithm, the percentage of inputs that will require more than the expected number of partitioning steps; a choice of $b = 4$ corresponds to a sensitivity of approximately 2%. The constant m is fixed and turns out to be 2 for sorting and $6/5$ for selection.

In our analysis, we assume that the original array length is n, pivots are selected using the median-of-3 strategy, and that given a pivot of rank p, the partitioning algorithm will produce two random subarrays of length $p-1$ and $n-p$. As usual, we consider the initial array to be a random permutation of distinct integers, all permutations being equally likely.

The sorting heuristic

For sorting, we focus on the size of the larger of the two subarrays produced following a partitioning step. As n becomes large, the expected ratio of the length of the larger subarray to the length of the

Copyright © 2000 John Wiley & Sons, Ltd.

original array is given by

$$\lim_{n\to\infty}\left(\frac{1}{n\binom{n}{3}}\sum_{p=2}^{n-1}\max(p-1,n-p)(p-1)(n-p)\right)=\frac{11}{16} \tag{1}$$

and the expected ratio of the square of the length of the larger subarray to the square of the length of the original is given by

$$\lim_{n\to\infty}\left(\frac{1}{n^2\binom{n}{3}}\sum_{p=2}^{n-1}\max^2(p-1,n-p)(p-1)(n-p)\right)=\frac{39}{80} \tag{2}$$

Let $\{X_1, X_2, \ldots\}$ be a set of pairwise independent random variables such that for each $i > 0$, $\mathrm{E}[X_i] = 11/16$ and $\mathrm{var}[X_i] = 39/80 - (11/16)^2$. Intuitively, each X_i represents the ratio by which the ith partitioning step will split the array. Note that after k partitions we have

$$\mathrm{E}\left[\prod_{i=1}^{k} X_i\right]=\left(\frac{11}{16}\right)^k \tag{3}$$

and

$$\mathrm{var}\left[\prod_{i=1}^{k} X_i\right]=\left(\frac{39}{80}\right)^k-\left(\frac{11}{16}\right)^{2k} \tag{4}$$

Therefore, by Chebyshev's inequality, $\prod_{i=1}^{k} X_i < 2^{-x}$ with probability at least $1-1/(d^2+1)$ where d, k and x satisfy

$$\left(\frac{11}{16}\right)^k+d\sqrt{\left(\frac{39}{80}\right)^k-\left(\frac{11}{16}\right)^{2k}}=2^{-x} \tag{5}$$

Given $x > 0$ and $d > 1$, we would like to find the value of k satisfying (5). Defining the fraction

$$f(k,x)=\frac{(11/16)^k}{2^{-x}}$$

we can manipulate (5) to obtain

$$\begin{aligned} d^2\left(\frac{39}{80}\right)^k-d^2\left(\frac{11}{16}\right)^{2k} &= 2^{-2x}-2^{1-x}\left(\frac{11}{16}\right)^k+\left(\frac{11}{16}\right)^{2k} \\ d^2\left(\frac{39}{80}\right)^k &= 2^{-2x}-2^{-2x}2f(k,x)+2^{-2x}(d^2+1)f^2(k,x) \\ d^2\left(\frac{39}{80}\right)^k &= 2^{-2x}((d^2+1)f^2(k,x)-2f(k,x)+1) \end{aligned} \tag{6}$$

Copyright © 2000 John Wiley & Sons, Ltd.

Table I. Confidence level and range of exact fit for the $2x + b$ heuristic.

b	d	Confidence	Up to
1	1.450	67.77%	$x = 22$
2	2.670	87.69%	27
3	4.279	94.82%	29
4	6.506	97.69%	30
5	9.627	98.93%	31
6	14.023	99.49%	31

The right-hand side of (6) is minimized by replacing $f(k, x)$ with $1/(d^2+1)$; this yields a lower bound for k of

$$d^2 \left(\frac{39}{80}\right)^k \leq 2^{-2x}(1 - 1/(d^2 + 1))$$
$$k \geq \frac{2 \ln 2}{\ln \frac{80}{39}} x + C$$
$$k \geq 1.93x + C \tag{7}$$

where C is a constant depending on d.

Recall that k is the number of partitioning steps we expect will reduce the size of the array to less than 2^{-x} times its original size. Equation (7) suggests the simple approximation

$$\hat{k} = 2x + b \tag{8}$$

where b is an integer constant depending on d, and hence the confidence level that $\hat{k}$ partitions will indeed suffice to reduce the array to the desired size. It turns out that for given values of b, we can find values for d such that $\hat{k} = \lceil k \rceil$ up to a reasonable value for x. Table I gives the appropriate value for d, the corresponding confidence level, and the value of x up to which the approximation is exact for selected values of b, computed numerically from Equation (5). For larger values of x the heuristic will give a value for $\hat{k}$ that is too large. However, the error grows slowly enough that for practical purposes it can be ignored; for example, for $b = 4$, $\hat{k} - \lceil k \rceil \leq 1$ up to $x = 45$.

The selection heuristic

For selection based on Hoare's Find algorithm, only one of the two subarrays is processed recursively following partitioning. If we are selecting the mth element of the array and p is the rank of the pivot, then the length of this subarray is given by the function

$$L_{n,m}(p) = \begin{cases} \min(p-1, n-p) & \text{if } p \text{ falls between } m \text{ and } \lceil n/2 \rceil \\ \max(p-1, n-p) & \text{otherwise} \end{cases} \tag{9}$$

Copyright © 2000 John Wiley & Sons, Ltd.

Table II. Confidence level and range of exact fit for the $\lfloor (6/5)x \rfloor + b$ heuristic.

b	d	Confidence	Up to
2	1.807	76.56%	$x = 19$
3	3.623	92.92%	29
4	6.716	97.83%	34
5	12.192	99.33%	39
6	21.916	99.79%	39
7	39.223	99.94%	39

Therefore, taking all values of m as being equally likely, the expected ratio of the length of the subarray following a partitioning step to the length of the original array is

$$\lim_{n\to\infty} \left(\frac{1}{n^2 \binom{n}{3}} \sum_{m=1}^{n} \sum_{p=2}^{n-1} L_{n,m}(p)(p-1)(n-p) \right) = \frac{41}{80} \tag{10}$$

and the expected ratio of the square of the length of the subarray to the square of the length of the original is

$$\lim_{n\to\infty} \left(\frac{1}{n^3 \binom{n}{3}} \sum_{m=1}^{n} \sum_{p=2}^{n-1} L_{n,m}^2(p)(p-1)(n-p) \right) = \frac{5}{16} \tag{11}$$

Proceeding as for the sorting case, we obtain

$$k \geq 1.19x + C \tag{12}$$

suggesting

$$\hat{k} = \lfloor (6/5)x \rfloor + b \tag{13}$$

as an approximation. Table II gives the parameters for selected values of b. There is no entry for $b = 1$ because the resulting value for d is less than one. For the values listed, $\hat{k} - \lceil k \rceil \leq 1$ up to at least $x = 124$.

IMPLEMENTATION WITH REMEDIAL RANDOMIZATION

The heuristics of the previous section provide a simple means of detecting potential worst-case inputs after only a small number partitioning steps. Early detection of worst-case inputs can result in a substantial improvement in the running time by switching to the stopper algorithm sooner and spending less time performing unproductive partitions. However, false positives can still cause an introspective algorithm to be slower than its standard counterpart for certain inputs.

Copyright © 2000 John Wiley & Sons, Ltd.

To minimize the performance impact of false positives, we need an alternative intervention strategy. A logical approach is to inject some amount of entropy into the array and allow the standard algorithm to continue. For false positives, this will not (unless we are very unlucky) impact the running time, while for true worst-case inputs it may be enough to disrupt the quadratic behavior and result in close to average-case behavior.

How much entropy should we add when introspection indicates a worst-case input? Shuffling the entire array is expensive and perhaps overkill. At the other extreme, we could simply select the next pivot randomly, guaranteeing a good pivot on average; however, we shall see in the next section that there exist cases where this will not disrupt the quadratic behavior of the standard algorithm, which will continue to select poor pivots. A compromise between the two is to swap the first, middle, and last elements with three randomly chosen elements in the array, resulting in a random pivot selection for the next partitioning step while adding a small amount of entropy.

The low cost of this approach works to our advantage; after a randomization step, we continue with the algorithm as usual, restarting the introspection as if this were a new array. If it turns out that the intervention was not enough to prevent quadratic behavior, another randomization step will be triggered shortly. We thus adaptively add randomization in small doses in order to counteract quadratic behavior, while incurring virtually no penalty for false positives.

Since, in the worst case, one pivot will be chosen at random every constant number of partitions, the running time of an introspective algorithm using remedial randomization is within a constant factor of the average case for the standard algorithm with high probability. If we desire an absolute worst-case bound that matches the average case, we can place a limit on the total number of randomization steps and switch to the stopper algorithm if this is exceeded.

Sorting implementation

Figure 1 presents a C++ implementation of a sorting algorithm incorporating both fine-grained introspection and remedial randomization, in addition to the use of Heapsort as a stopper to guarantee $\Theta(n \log n)$ worst-case running time. The implementation is based on that of the `sort` function in the Hewlett-Packard reference implementation of the Standard Template Library (STL) [5].

The algorithm can be tuned by setting three constants: `b`, which should be set to an appropriate value of b from Table I; `coef`, which should be set to $2c$ for some constant $c > 0$; and `div`, which should be set to 2^c. By choosing a larger value for c, the overhead of introspection can be reduced, and by choosing a larger value for b, the sensitivity of the introspection can be reduced.

Apart from the introspection, the code itself differs little from a typical optimized Quicksort implementation. The main sorting loop takes as parameters two random access iterators [6] describing the interval of the array to sort. Three variables are used to drive the introspection: `counter`, which counts down the number of partitioning steps left before the size of the array is checked; `threshold`, the maximum expected size of the array when `counter` reaches zero; and `limit`, which is used to limit the total number of randomization steps before resorting to the stopper algorithm.

Arrays of a given length (here 16) or less are left for a final insertion sort pass [7]. When it is time for an introspective check, the size of the array is compared to the threshold value; if it is too large, a randomization step is performed, swapping the first, middle, and last elements with three randomly chosen elements in the array. The number of randomization steps is limited to $\lfloor \log_2 n \rfloor$, and if this limit

Copyright © 2000 John Wiley & Sons, Ltd.

```
1  template <class Iterator>
2  void sort_loop(Iterator first, Iterator last)
   {
3    int counter = b + coef + 1;
4    unsigned threshold = (last - first) / div;
5    unsigned limit = (last - first);

6    while (last - first > 16) {
7      if (--counter == 0) {
8        if (last - first >= threshold) {
9          if (limit /= 2) {
10           iter_swap(first, first + rng(last - first));
11           iter_swap(last - 1, first + 1 + rng(last - first - 1));
12           iter_swap(first + (last - first) / 2,
                           first + 1 + rng(last - first - 2));
13           counter = b + coef;
14           threshold = (last - first) / div;
           } else {
15           heapsort(first, last);
16           break;
           }
         } else {
17         counter = coef;
18         threshold /= div;
         }
       }

19     typename iterator_traits<Iterator>::value_type pivot
        = median(*first, *(first + (last - first)/2), *(last - 1));
20     Iterator cut = partition(first, last, pivot);
21     if (cut - first >= last - cut) {
22       sort_loop(cut, last);
23       last = cut;
       } else {
24       sort_loop(first, cut);
25       first = cut;
       }
     }
   }
```

Figure 1. Introspective sorting algorithm employing fine-grained introspection, remedial randomization, and Heapsort as a stopper.

Copyright © 2000 John Wiley & Sons, Ltd.

is exceeded, Heapsort is used to finish sorting the array, guaranteeing $\Theta(n \log n)$ running time in the worst case.

Provided the stopper algorithm is not invoked, the control variables are updated and the algorithm proceeds as in a typical Quicksort implementation: the pivot is selected as the median of the first, middle, and last elements; the array is partitioned; and the smaller half of the array is recursively sorted before looping to process the larger half. Note that this ensures that the introspection continues on the larger half, while it is restarted from the beginning for the smaller half.

Introspection adds considerably to the bulk of the code. However, for well-behaved inputs, most of the additional code will never be executed, and the small amount of overhead to initialize the control variables and perform the checking at lines 7 and 8 is negligible compared to the amount of time spent in the partitioning subroutine. Our experimental results show that for well-behaved inputs there is virtually no difference between the performance of the algorithm in Figure 1 and that of a standard Quicksort implementation without introspection.

Selection implementation

Figure 2 presents a C++ implementation of a selection algorithm based on the same principles as the sorting algorithm of the previous section. The implementation is based upon that of the `nth_element` function in the Hewlett-Packard reference implementation of the STL.

The algorithm can be tuned by setting the constant `b`, which should be set to an appropriate value from Table II. Our implementation is slightly complicated by the non-integral slope of the heuristic; the loop at lines 6–9 performs the initial partitions, and thereafter the loop at lines 11–24 performs an introspective check followed by a partition, skipping the check every five iterations. As with the sorting algorithm, following a randomization step the introspection is restarted.

To guarantee a linear running time in the worst case, we cannot use the algorithm's depth of recursion [3]. Instead, we follow Musser's suggestion and limit the sum of the length of each subarray following a randomization step. After partitioning this is expected to be about one-half its prior length, so an obvious approach is to limit the sum to twice the original length of the array. We approximate this by subtracting one-fourth of the length of the subarray prior to partitioning from the original array length.

One implementation concern for both sorting and selection algorithms is the choice of pseudo-random number generator for the randomization steps. As a practical matter, the state of the RNG should be private to the algorithm [8]; and while performance is not overly critical, since it will be negligible compared to the time taken for the unproductive partitions that resulted in randomization being triggered, it should be reasonably fast, preferably not requiring a subroutine call. Since the number of randomization steps is limited by the length of the input array, the period of the generator need not be extraordinary. Probably most important is that the method used should not require anything in the way of initialization prior to invocation of the algorithm proper. A simple linear congruential generator [9] meets these criteria and performed well in our tests.

EXPERIMENTAL RESULTS

We performed a series of experiments in order to evaluate the effect of the techniques described in this paper on the performance of sorting and selection algorithms. In addition to the running time, we

Copyright © 2000 John Wiley & Sons, Ltd.

```
1  template <class Iterator>
2  void select_loop(Iterator first, Iterator nth, Iterator last)
   {
3    unsigned limit = last - first;

   begin:
4    unsigned threshold = (last - first) / 2;
5    int counter = b + 2;
6    while (last - first > 3 && --counter) {
7      typename iterator_traits<Iterator>::value_type pivot
        = median(*first, *(first + (last - first)/2), *(last - 1));
8      Iterator cut = partition(first, last, pivot);
9      if (cut <= nth) first = cut; else last = cut;
     }
10   counter = -1;
11   while (last - first > 3) {
12     if (--counter < 0) {
13       if (last - first >= threshold) {
14         if ((last - first) / 4 <= limit) {
15           limit -= (last - first) / 4;
16           iter_swap(first, first + rng(last - first));
17           iter_swap(last - 1, first + 1 + rng(last - first - 1));
18           iter_swap(first + (last - first) / 2,
                          first + 1 + rng(last - first - 2));
19           goto begin;
           } else {
20           linear_select(first, nth, last);
21           return;
           }
         } else {
22         if (counter == -5)
23           counter = 1;
24         threshold /= 2;
         }
       }

25     typename iterator_traits<Iterator>::value_type pivot
        = median(*first, *(first + (last - first)/2), *(last - 1));
26     Iterator cut = partition(first, last, pivot);
27     if (cut <= nth) first = cut; else last = cut;
   }
   }
```

Figure 2. Introspective selection algorithm employing fine-grained introspection, remedial randomization, and the linear time selection algorithm as a stopper.

Copyright © 2000 John Wiley & Sons, Ltd.

Table III. Sorting and selection algorithms evaluated.

Algorithm	Description
hpsort	A standard Quicksort implementation based on the HP reference implementation of the STL.
heapsort	Heapsort, as implemented by the `std::partial_sort` function.
introsort	Musser's Introsort, using depth-limiting introspection and Heapsort as a stopper algorithm.
risort	The algorithm of Figure 1, with $\mathtt{b} = 4$ and $\mathtt{coef} = \mathtt{div} = 2$, using fine-grained introspection, remedial randomization, and Heapsort as a stopper.
isort	The same algorithm as risort above but without remedial randomization; it switches to Heapsort as soon as fine-grained introspection detects a potential worst case.
hpselect	A standard Find implementation based on the HP reference implementation of the STL.
linear_select	The linear time selection algorithm of Blum *et al.* [4].
riselect	The algorithm of Figure 2, with $\mathtt{b} = 4$, using fine-grained introspection, remedial randomization, and the linear selection algorithm as a stopper.
iselect	The same algorithm as riselect above but without remedial randomization; it switches the linear time algorithm as soon as a potential worst case is indicated.

measured the number of various types of operations using object adaptors for counting comparisons, assignments, and iterator operations [3,10], the number of randomization steps performed (where applicable), the total number of partitions performed, and whether or not the stopper algorithm was invoked.

We evaluated the algorithms listed in Table III. In order to make the comparisons fair, all of the Quicksort and Find variants share the same common partitioning code, derived from the original STL implementation [5]. The Heapsort implementation used was that provided by the C++ standard library function `partial_sort`, which in many implementations is derived from the same source. Presumably these codes have been carefully optimized, and apart from the introspection itself, we did not attempt any additional optimization.

The results reported here were obtained on a MIPS R5000 processor with 512 kilobytes of secondary cache and 64 megabytes of main memory, using version 7.2.1 of the Silicon Graphics MIPSpro C++ compiler. Similar results were obtained on Pentium and SPARC processors, using the Microsoft version 5.0 and GCC version 2.8.1 compilers.

We evaluated the algorithms using a range of different input sequences, listed in Table IV, designed to test the performance of the algorithms under a range of conditions. The common cases of the random, sorted, and onezero (representing inputs containing many duplicates) sequences should all be handled with virtually no additional overhead over the standard non-introspective algorithm. The m3killer [3] and rotated [11] sequences cause non-introspective algorithms based on median-of-3 pivot selection

Copyright © 2000 John Wiley & Sons, Ltd.

Table IV. Input sequences used to evaluate algorithms.

Sequence	Description
random	A random permutation of the integers 0 through $n-1$.
sorted	The integers 0 through $n-1$ in increasing order.
onezero	A random permutation of $\lfloor n/2 \rfloor$ ones and $\lceil n/2 \rceil$ zeroes.
m3killer	Musser's 'median-of-3 killer' sequence, obtained by applying the following two permutations to a sorted sequence of length $n = 4k$: $\begin{pmatrix} 1 & 2 & 3 & 4 & \dots & 4k-1 & 4k \\ 1 & 2k+1 & 2 & 2k+2 & \dots & 2k & 4k \end{pmatrix}$ followed by $\begin{pmatrix} 1 & 2 & \dots & k & k+1 & k+2 & \dots & 2k & 2k+1 & \dots & 4k \\ 1 & 3 & \dots & 2k-1 & 2 & 4 & \dots & 2k & 2k+1 & \dots & 4k \end{pmatrix}$
rotated	A sorted sequence rotated left once; i.e. $\begin{pmatrix} 1 & 2 & \dots & n-1 & n \\ 2 & 3 & \dots & n & 1 \end{pmatrix}$.
twofaced	Musser's 'two-faced' sequence, obtained by randomly permuting the elements of a median-of-3 killer sequence in positions $4\lfloor \log_2 n \rfloor$ through $n/2$ and $n/2+4\lfloor \log_2 n \rfloor$ through n.
organpipe	The integers 1 through $n/2$ in increasing order, followed by $n/2$ through 1 in decreasing order, where n is even.

to exhibit worst-case performance, the latter sequence for the particular partitioning algorithm used in many implementations of the STL. The twofaced and organpipe sequences are examples of inputs that generate false positives for depth-limiting introspection; although they are not worst case, they require more than the average number of partitioning steps.

In addition, we evaluated the algorithms using Bentley and McIlroy's sorting performance profile [8], and two inputs designed to test the performance of introspective algorithms under worst-case conditions. McIlroy's 'Killer Quicksort' adversary [12] produces quadratic behavior in any Quicksort or Find variant, even those that choose pivots at random. We also simulate a more reasonable adversary which causes every deterministic pivot selection to be worst case, but can be overcome using randomization.

Methodology

For each pair of algorithm and input sequence from Tables III and IV, we ran a series of trials, varying the input size from 52 to 1 000 000. For combinations resulting in quadratic running time, the largest input size was limited to 50 000. For the random, onezero, and twofaced sequences, each trial used a

Copyright © 2000 John Wiley & Sons, Ltd.

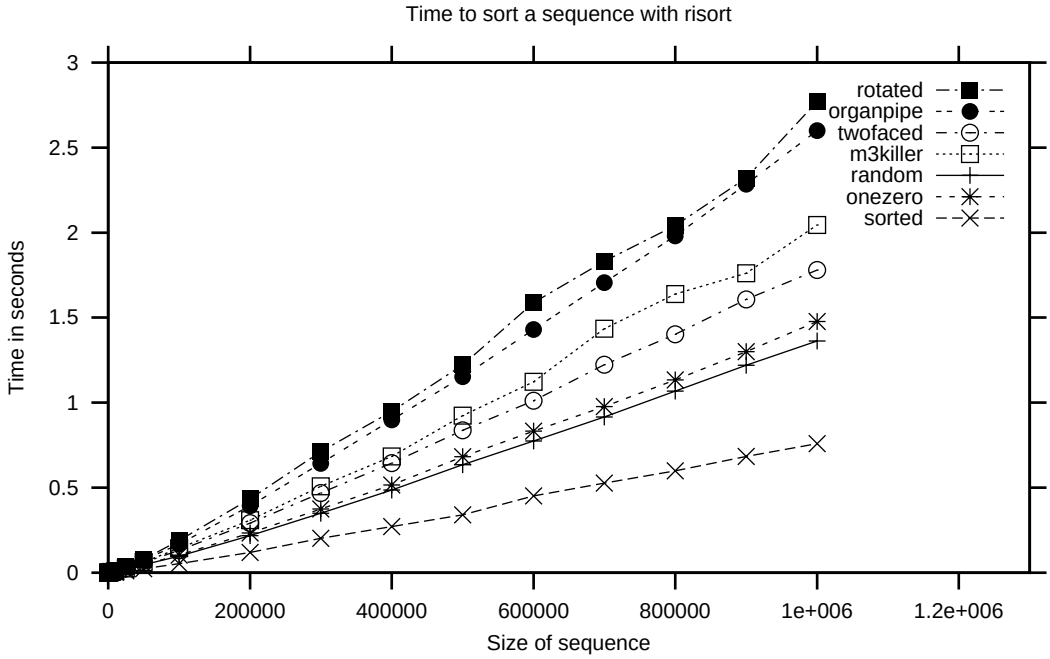


Figure 3. Performance of the sorting algorithms using fine-grained introspection and remedial randomization on the tested sequences.

different randomly generated input. For selection algorithms, the median element of the input array was found.

A minimum of five trials were performed for each pair of algorithm and sequence, except when measuring the operation counts for the deterministic sequences (sorted, m3killer, rotated, and organpipe), when only a single trial was required. Otherwise, trials were performed until a Student's t-test estimated the mean value of all the measures of interest to be statistically significant within plus or minus 10%, or until a maximum of 30 trials was reached. An exception to this was made for series in which some of the trials resulted in the stopper being invoked while others did not (e.g., the isort algorithm with the random sequence), for which we would expect the distribution to be bimodal.

Two synchronized pseudo-random number generators distinct from those used in the risort and riselect algorithms were used for the operation count and timing trials to ensure that the same sequences were used for both. All of our results were checked using two different pseudo-random number generation algorithms. Both the code used in the experiments and the raw data reported in this paper, as well as further graphs, have been made available [13].

Copyright © 2000 John Wiley & Sons, Ltd.

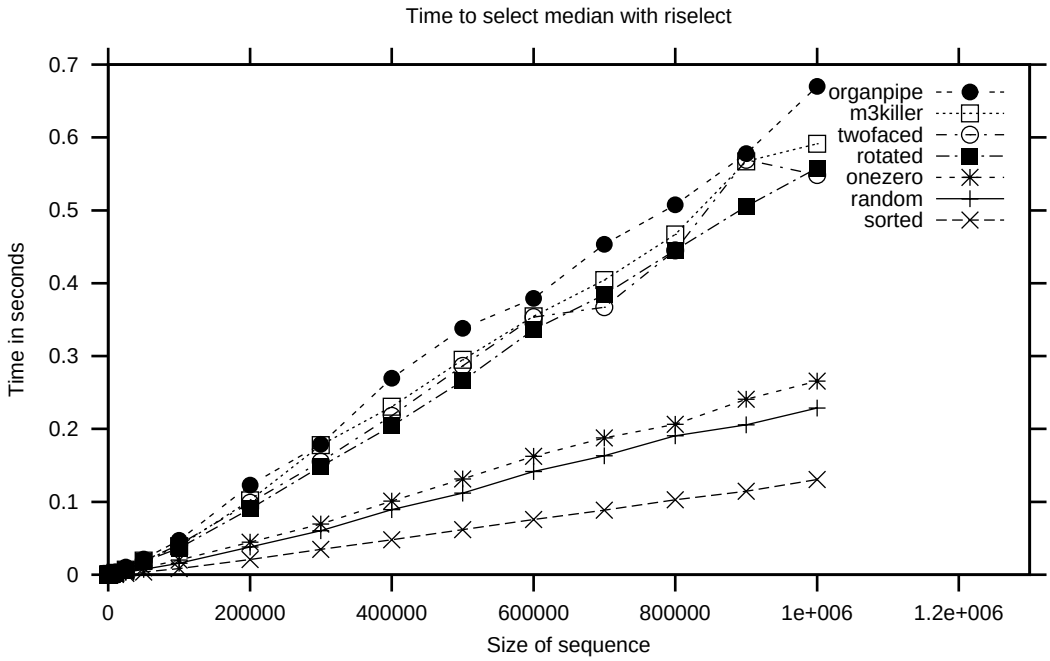


Figure 4. Performance of the selection algorithms using fine-grained introspection and remedial randomization on the tested sequences.

Results

For the random, sorted, and onezero sequences, we observed a clear performance gain from using one of the variants of Hoare's algorithms over Heapsort and the linear selection algorithm, as much as a factor of nine times faster. Furthermore, we observed virtually no difference in performance between the standard algorithms and any of the introspective variants.

Interestingly, although we observed that the stopper for isort and iselect is almost always triggered at some point in the execution of the algorithm on random inputs, their performance does not appear to suffer. We conjecture that this is because the stopper is not being triggered until the subarray has been reduced to a small size.

For the m3killer and rotated sequences, early detection using fine-grained introspection results in a significant performance boost, isort being only slightly slower than heapsort. Moreover, the addition of remedial randomization provides even more of an increase in performance.

For the twofaced and organpipe sequences, it is worth noting that even though neither is a worst-case input, risort enjoys a significant performance improvement due to the randomization steps. The stopper is invoked by isort on both, resulting in performance slightly worse than heapsort. This use of the stopper gives better performance for the organpipe sequence, for which hpsort performs relatively

Copyright © 2000 John Wiley & Sons, Ltd.

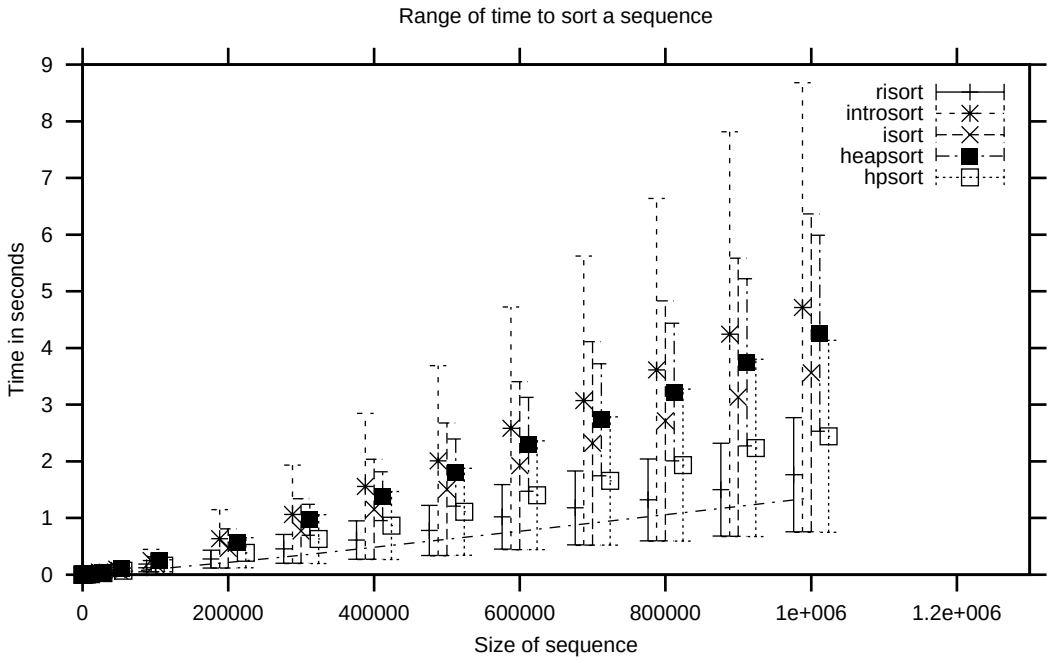


Figure 5. The range of performance for the sorting algorithms on the tested sequences.

poorly (although it does not go quadratic), but not for the twofaced sequence, where hpsort beats all of the other algorithms except for risort.

Figures 3 and 4 provide a consolidated view of the performance of risort and riselect on all of the sequences tested. Note that we have maintained the property that 'easy' sequences are processed quickly. The time to process 'hard' sequences is within a factor of three times that required for random inputs. For some inputs, for example the organpipe sequence, this is much better than the performance of either the standard or the stopper algorithm.

Figures 5 and 6 provide a comparison of the range of performance on all the sequences tested. We have omitted worst-case sequences from the range of hpsort and hpselect. The lines in each figure plot the performance of hpsort and hpselect, respectively, on the random sequence. Notice that the high end of the range for risort and riselect never exceeds the low end of the range for the stopper algorithm, and is significantly below the high end for any other variant.

Bentley and McIlroy [8] present a certification program for Quicksort that generates a 'performance profile' for an algorithm over a variety of inputs. Inputs are generated by varying four parameters: the length of the sequence, the base sequence type, an additional integer parameter used in the generation of the sequence, and an optional post-processing algorithm that further permutes the input ordering.

Copyright © 2000 John Wiley & Sons, Ltd.

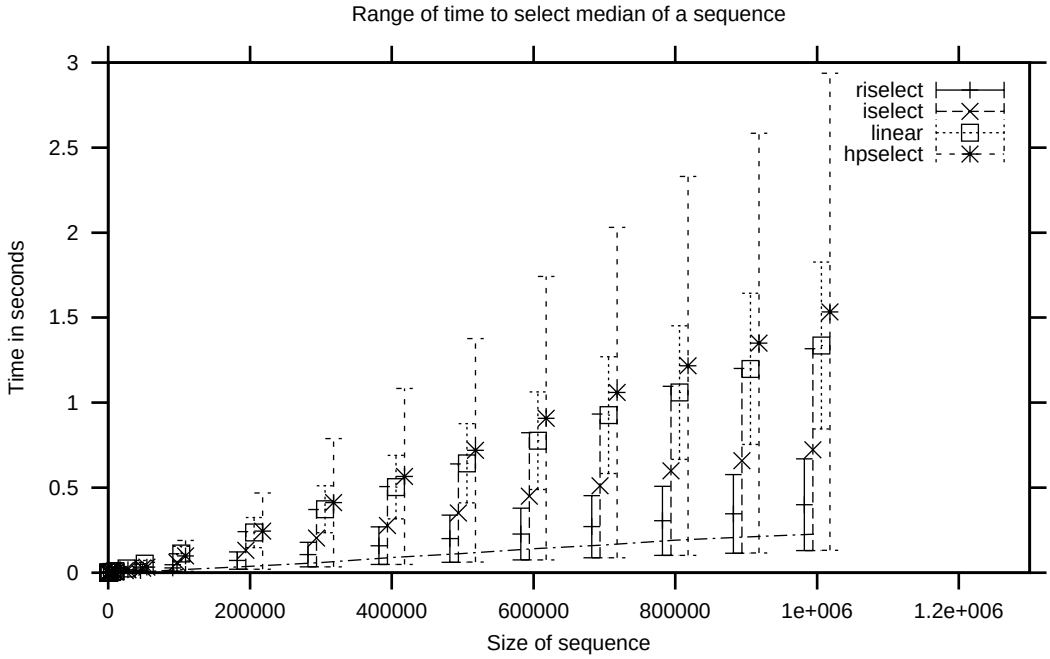


Figure 6. The range of performance for the selection algorithms on the tested sequences.

Algorithms are tested on inputs at four sizes (100 and 1023–1025), and are evaluated based on the total number of comparisons performed.

Figures 7 and 8 show the results for the algorithms we tested, each point representing the results for a single input at or near length 1024. We plot running time as opposed to just comparisons, and test selection algorithms (finding the median) in addition to sorting.

The results for hpsort and hpselect are omitted and contain a number of outliers; these are inputs that result in quadratic performance and are all variations on the rotated sequence. For example, a 'stagger' sequence with length $2^{k-1}+1$ and parameter 2^k for any $k \geq 1$ will produce a reverse sorted sequence rotated to the right once. Our results above are confirmed by the Bentley–McIlroy performance profile, showing a clear benefit to fine-grained introspection with remedial randomization.

Worst-case performance

The preceding results show that the techniques suggested in this paper produce sorting and selection algorithms that provide superior performance over a wide range of inputs. A natural question is, must this always be the case? In particular, we may question whether or not remedial randomization will always succeed in introducing sufficient entropy into the input to avoid poor pivot selection by the

Copyright © 2000 John Wiley & Sons, Ltd.

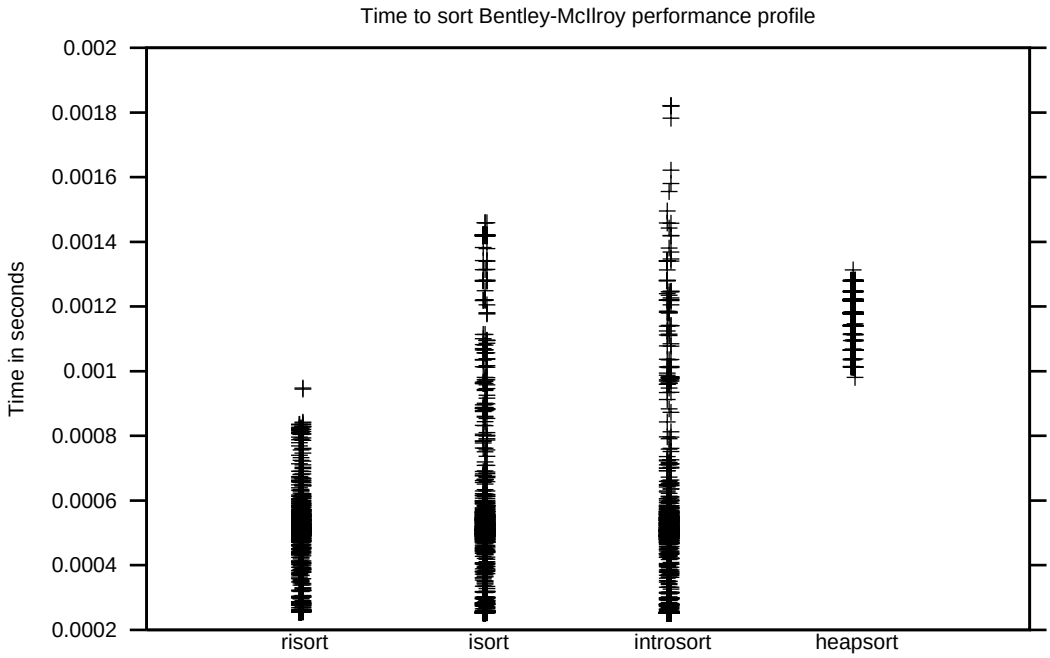


Figure 7. Sorting algorithm performance, Bentley–McIlroy profile ($n \approx 1024$).

median-of-3 strategy. While we are not able to answer this question directly, we can provide an idea of what the performance of algorithms using our techniques would be in a worst-case scenario.

McIlroy [12] describes an adversary that will elicit worst-case behavior from any Quicksort implementation, by constructing the input in an on-line fashion such that a poor pivot is always selected. Figures 9 and 10 show the results of running the tested algorithms against this adversary.

McIlroy's adversary is extremely strong; due to its on-line nature, it will confound even algorithms that choose pivots at random. In particular, the algorithms using remedial randomization are penalized because they 'waste' time with unproductive random pivot selections rather than switching immediately to the stopper. Despite this, the performance of riselect and risort is within a factor of four to six times that of the stopper.

Such a strong adversary is unrealistic; there is no reason to expect that random pivot selection will not produce good performance on average. However, we may still question whether or not the entropy introduced in a remedial randomization step will always guarantee that future deterministic pivot selections will be good.

In addition to causing the median-of-3 strategy to choose a worst case pivot, the rotated sequence has the property that any pivot selection will result in two rotated subarrays following partitioning. This regenerative behavior is why it is important for remedial randomization to actually swap randomly

Copyright © 2000 John Wiley & Sons, Ltd.

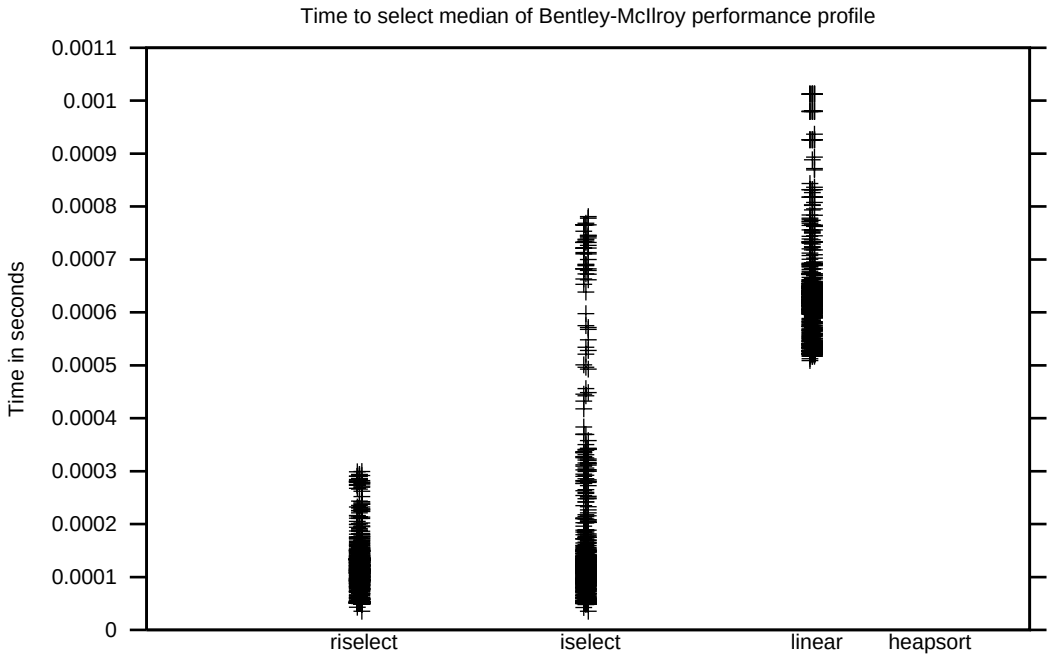


Figure 8. Selection algorithm performance, Bentley–McIlroy profile ($n \approx 1024$).

chosen elements into position, rather than simply choosing a pivot at random; otherwise, even though the random pivot will produce a good split on average, subsequent deterministic choices will continue to be worst-case. By skipping the swapping step, we can simulate an adversary that always forces a poor choice for deterministic pivot selection, but allows random selection to produce good pivots.

Figures 11 and 12 show the behavior of sorting and selection algorithms for this simulated adversary. The performance of risort is comparable to introsort, and within a factor of two times that of heapsort, and for selection the penalty for failed remedial randomization is even smaller. It is worth pointing out that the rotated sequence is, for heapsort, a near best-case input; random inputs will take more than twice as much time. We also found that while risort does about half as many total operations as the other algorithms, most of these are comparisons done during unproductive partitions, as opposed to cheaper iterator operations.

Use of randomization

We would expect that some inputs will be more difficult for remedial randomization to 'fix' than others. One way of measuring this is to look at the number of randomization steps taken by risort and riselect for a given sequence.

Copyright © 2000 John Wiley & Sons, Ltd.

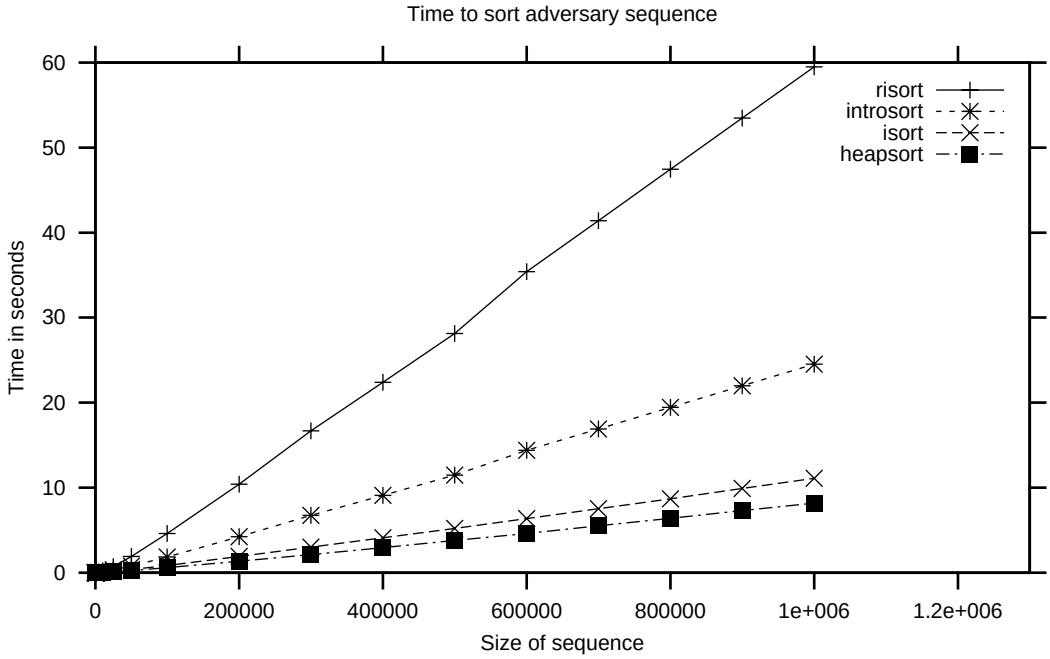


Figure 9. Performance of the sorting algorithms against McIlroy's 'Killer Quicksort' adversary.

The simulated 'worst-case' adversary input described above requires the most randomization steps. In fact, it produces the maximum number of randomization steps possible for any input, around 13–14% of the total number of partitions. McIlroy's adversary does not produce nearly so many, because, by confounding even random pivot selections, it causes the stopper to be invoked much sooner. There is a comfortable margin between this maximum and the number of randomization steps required by the rotated sequence; however this sequence does require significantly more than any other sequence we tested, about 10% of all partitions. The reason for this is that any permutation of the sequence that does not introduce a significant amount of entropy tends to result in an embedded rotated subsequence in one or both subarrays. For selection, the rotated sequence does not pose nearly so much of a problem for remedial randomization, because with some constant probability the algorithm will continue with a subarray that does not contain a rotated subsequence.

The m3killer and organpipe sequences both end up requiring about 3% of the partitions to be randomization steps; for the random and onezero sequences the figure is only a few hundreths of one percent. Since McIlroy's adversary causes every pivot chosen to be worst case, it forces the algorithm to resort to the stopper after exactly $\lfloor \log_2 n \rfloor$ partitions.

The Quicksort variants we tested will do at least $n/16$ partitions, and will do about $n/9$ on average [14]. Table V gives the approximate ratio of the input size to the number of randomization

Copyright © 2000 John Wiley & Sons, Ltd.

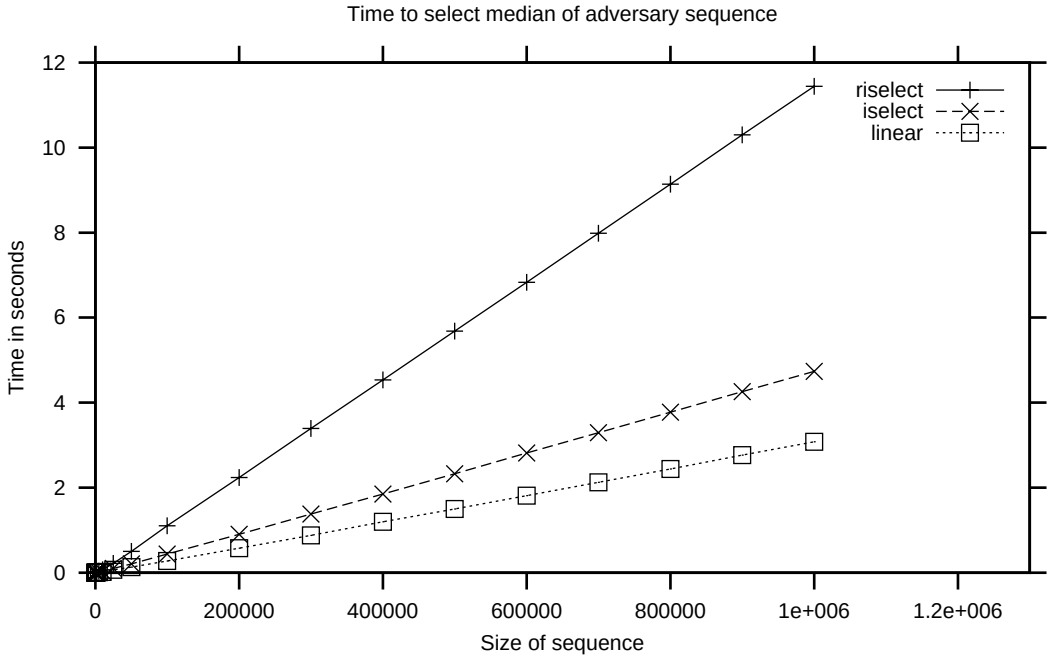


Figure 10. Performance of the selection algorithms against McIlroy's 'Killer Quicksort' adversary.

steps performed by risort for selected inputs, found using a least-squares fit to the data. The value for the worstcase sequence is exactly what we would expect, since only randomization step partitions are productive, and we limit the total to half of what would be required to sort the array before resorting to the stopper.

DISCUSSION

We have suggested two new techniques for sorting and selection algorithms, fine-grained introspection and remedial randomization, and shown that they provide significant performance improvements.

Our techniques can be viewed as a hybrid between always choosing pivots randomly, versus always using a deterministic technique. The number of random pivot selections is determined adaptively, responding to how ill-behaved the input is. Doing so only when the deterministic strategy performs poorly both avoids the objection that random pivot selection is too time consuming [3], and also does not sacrifice performance on best-case sorted or nearly sorted inputs, as would the strategy of simply always choosing the pivot at random. Users expect 'easy' cases to sort quickly [8], and in our

Copyright © 2000 John Wiley & Sons, Ltd.

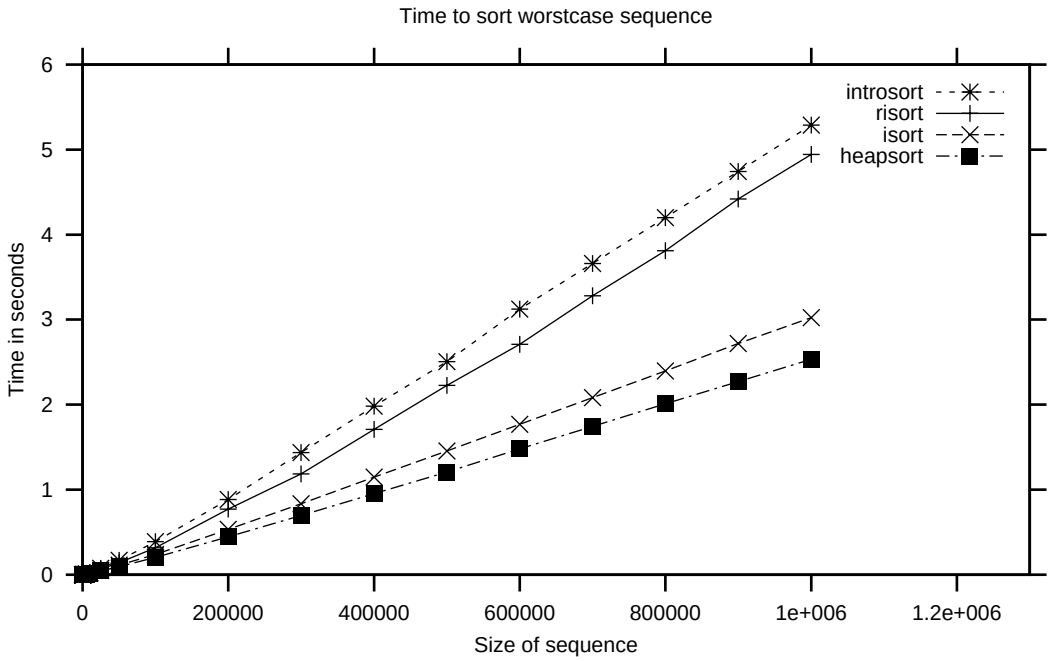


Figure 11. Performance of the sorting algorithms on 'worst-case' sequence.

experiments the median-of-3 strategy performs nearly twice as fast on sorted sequences compared to random inputs.

Clearly the rotated sequence is an exceptionally difficult input; this is somewhat surprising since it is very nearly already sorted. As noted previously, this behavior only occurs for the specific partitioning algorithm used. A variation of this algorithm that does not induce worst-case performance on the rotated sequence sorts the first, middle, and last elements of the array into the first, second, and last positions, and places the pivot element into its correct sorted position following partitioning [7].

Using this 'sorted-median' partitioning algorithm in our experiments resulted in slightly worse performance when sorting already sorted or m3killer inputs, but better performance on other inputs. For the common case of random inputs, the improvement was moderately significant, and for selection, sorted-median partitioning always resulted in the same or better performance. On balance, it appears worthwhile to adopt this algorithm.

We chose the tuning constants used in the implementation of risort and riselect ($\texttt{b} = 4$, $\texttt{coef} = 2$, and $\texttt{div} = 2$) more or less arbitrarily; future work should experiment to determine what the optimal settings are.

Musser [3] has pointed out that a possible objection to introspective methods is the increased object code footprint due to the inclusion of the stopper algorithm. If we are willing to forego the absolute

Copyright © 2000 John Wiley & Sons, Ltd.

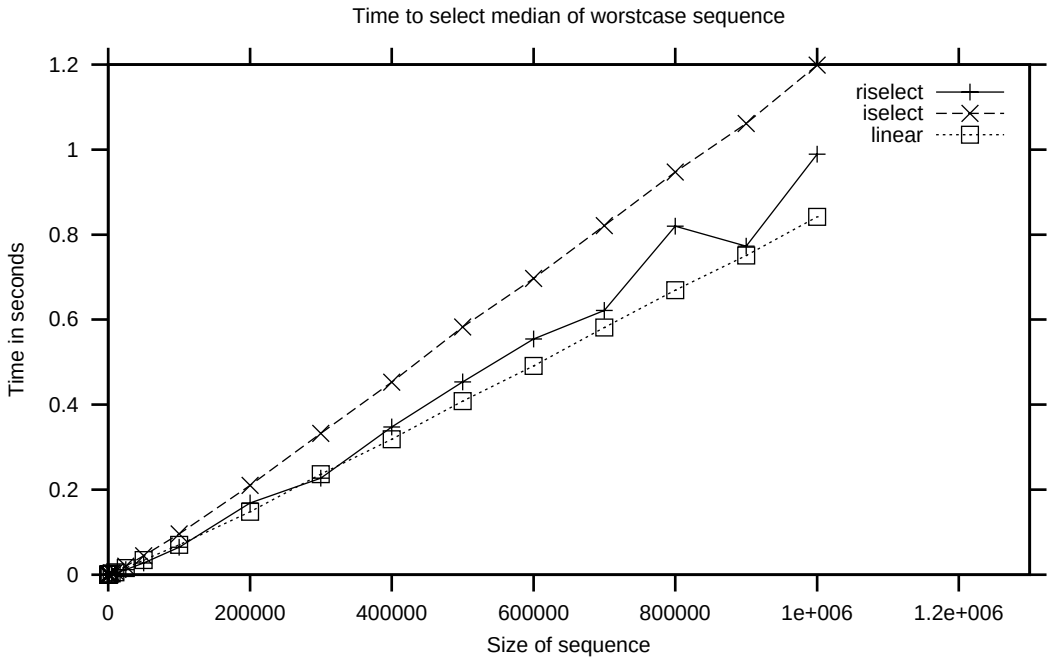


Figure 12. Performance of the selection algorithms on 'worst-case' sequence.

Table V. Approximate ratio of input size to number of randomization steps.

Sequence	Ratio
worstcase	18
rotated	43
organpipe	251
m3killer	252
twofaced	32 415
random	35 002

performance guarantee and instead settle for a high probability guarantee, we can omit the stopper algorithm entirely and rely solely on remedial randomization steps in the worst case, eliminating the code-size penalty. In our experiments, we did not observe the stopper algorithm being invoked by either risort or riselect, except for the pathological adversary and worst-case sequences.

Copyright © 2000 John Wiley & Sons, Ltd.

For sufficiently large inputs, the heuristics used for fine-grained introspection no longer accurately give the expected number of partitioning steps required to reduce the size of the array by half. However, the heuristic always errs on the side of too many partitions, and the worst that can happen is that some small performance penalty may be incurred due to a worst-case input being detected a few partitioning steps later than it might have been.

Quicksort has been characterized as a gamble [15], a statement underscored by the fact that worst-case behavior continues to crop up even in careful implementations (cf. [8,11]). Introspective algorithms provide insurance against quadratic behavior; the techniques described in this paper allow this insurance to be obtained at practically no cost, and in addition improve performance in instances that are not necessarily worst case, but nonetheless are worse than average (e.g., the organpipe sequence).

ACKNOWLEDGEMENTS

D. Musser and K. Valois made valuable comments on earlier drafts of this paper. Source code for the experiments was contributed by D. Musser and A. Shapira.

REFERENCES

1. Hoare CAR. Quicksort. *Computer Journal* 1962; **5**:10–15.
2. Singleton RC. Algorithm 347: An efficient algorithm for sorting with minimal storage. *Communications of the ACM* 1969; **12**:185–187.
3. Musser DR. Introspective sorting and selection algorithms. *Software—Practice and Experience* 1997; **27**(8):983–993.
4. Blum M, Floyd RW, Pratt V, Rivest RL, Tarjan RE. Time bounds for selection. *Journal of Computer and System Sciences* 1973; **7**(4):448–461.
5. Stepanov AA, Lee M, Musser DR. *Hewlett-Packard Laboratories reference implementation of the Standard Template Library*. ftp://ftp.cs.rpi.edu/pub/stl [7 January 1996].
6. *ISO/IEC International Standard 14882: Programming Languages—C++*, (1st edn); International Organization for Standardization, 1998; 513–514.
7. Sedgewick R. Implementing quicksort programs. *Communications of the ACM* 1978; **21**:847–856.
8. Bentley JL, McIlroy MD. Engineering a sort function. *Software—Practice and Experience* 1993; **23**(11):1249–1265.
9. Knuth DE. *Seminumerical Algorithms*; Addison-Wesley: Reading, MA, 1973.
10. Musser DR. Measuring computing times and operation counts of generic algorithms. http://www.cs.rpi.edu/~musser/gp/timing.html [19 September 1996].
11. Talvola G. Poor quicksort implementation? http://www.deja.com/getdoc.xp?AN=366009862 [25 June 1998].
12. McIlroy MD. A killer adversary for quicksort. *Software—Practice and Experience* 1999; **29**(4):341–344.
13. Experimental code, results, and graphs. ftp://ftp.cs.rpi.edu/pub/valoisj/risort [26 July 1999].
14. Knuth DE. *Sorting and Searching*; Addison-Wesley: Reading, MA, 1973.
15. Wirth N. *Algorithms + Data Structures = Programs*; Prentice-Hall: Englewood Cliffs, NJ, 1976.

Copyright © 2000 John Wiley & Sons, Ltd.